Information Security

Lecture 2b: Stateless Tracking and Browser Defences

A textbook treatment

Based on the lectures, TPs and quiz of Imane Fouad

October 25, 2026

Contents

How to read this book **2**

3 Stateless Tracking and Browser Defences **3**
3.1 Recognition Without State 3
3.2 Entropy: How Much a Feature Identifies 4
3.3 Attributes and Their Physics 5
3.4 Uniqueness, Linkability, Consistency 7
3.5 Blocking: Filter Lists and Behaviour 8
3.6 Browsers Against Tracking 9
Chapter takeaways 11

How to read this book

Each concept in this book is presented in the same rhythm, and every section closes with a *Go deeper* box (the exact details, numbers and edge cases a demanding examiner asks about) followed by *Hard questions, answered.*

1. **Motivation and intuition.** Why do we care? What problem does the idea solve, and what picture should you hold in your head?
2. **Formal statement.** The precise definition or theorem, set in a coloured box so it is easy to find when revising.
3. **Proof.** Every claim is proved. Longer proofs are split into labelled steps, and the one idea that makes the proof work is isolated in a *Key insight* box.
4. **Interpretation and discussion.** What the result really says, where it applies, and which misreadings to avoid.
5. **Examples.** From "trivially small" to "genuinely useful."
6. **Techniques, classic problems and gotchas.** Computational shortcuts, proof patterns, exam favourites with one-line solutions, and the mistakes everyone makes once.

Margin notes carry the informal, "say it out loud" version of what is happening in the main text. Every major section opens with a *What are we trying to get?* box that states the goal in one breath before any mathematics, and closes with a summary box.

Prerequisites. Comfort with binary and hexadecimal, elementary probability, and the modular arithmetic of integers; everything else (finite fields for AES, the number theory behind RSA, Shannon entropy) is built where it is needed.

Chapter 3

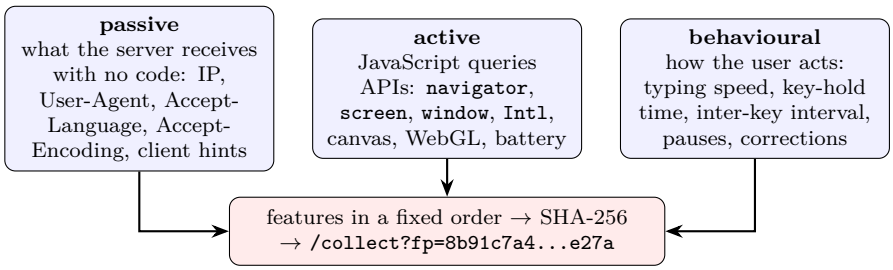
Stateless Tracking and Browser Defences

3.1 Recognition Without State

What are we trying to get?

Chapter 2 ended with a profile that survived the deletion of every store. We want the mechanism that explains it: recognising a browser from what it *reveals on every visit*, with no identifier stored; and the three ways that information is collected.

Definition 3.1.1 (Fingerprint; stateful vs stateless)**.** A **browser fingerprint** is the set of information about a device, from its hardware through its operating system to the browser and its configuration, that a web page can observe; **fingerprinting** is collecting it to recognise the device. **Stateful** tracking stores an identifier and reads it back later (cookies, storage, advertising IDs): "I recognise you because I stored something". **Stateless** tracking recomputes a recognisable pattern from properties exposed *now*: "I recognise you from what you reveal each time". Clearing state removes the stored identifier; it does not stop the pattern from being recomputed.



Reading the formula

The test for stateless is *where the identifier comes from*: an ETag is stateful (it sits in the cache) though it is not a cookie; a canvas hash is stateless though it looks like an ID. The three collection modes differ in what the tracker must do: passive costs nothing and needs no script; active needs JavaScript and reads APIs that the request never carried (screen size, fonts, time zone); behavioural measures the *user* rather than the device and is probabilistic (keyboard, fatigue, stress change it). Hashing the features in a fixed order

gives one short identifier: the same combination always maps to the same value, comparisons are cheap, and the user sees an unreadable string instead of the raw features, which is why fingerprinting is hard to notice.

The attack surface grew with the browser: Netscape Communicator 5.0 (1998, first open-source browser, ≈742k lines of code) to Chromium 120 (January 2024, ≈31M lines). Each new API (GPU, gyroscope, accelerometer, gamepad, USB, Bluetooth, AR/VR) is a feature and an attribute. A 1995 page learned "Netscape, French"; a 2024 page learns OS, screen, language, time zone, graphics card, audio and video capabilities.

Go deeper: the details an examiner asks about

- A fingerprint identifies a *browser instance*, never a named person (Quiz 1 Q11A false); exact matches produce false positives.
- Quiz 1 Q13: before any JavaScript runs, the server observes IP, User-Agent, Accept-Language, Accept-Encoding and client hints directly (A); screen resolution and fonts need active collection (B false); standard headers do distinguish browsers (C false); no cookie is needed first (D false).
- Disabling JavaScript stops active and behavioural collection; passive headers and stateful mechanisms remain.

Hard questions, answered

Q. Why is a hashed fingerprint not a permanent identifier?
A. Any changed feature (a browser update alters the User-Agent, a new font appears) changes the hash; following a browser over time needs *linking* (§3.4), not exact matching.

Q. Alice blocks cookies and uses private mode. Which of the three modes still work unchanged?
A. All three: none of them stores anything. Only tools that spoof or randomise attributes change the fingerprint, and those are themselves detectable (§3.4).

3.2 Entropy: How Much a Feature Identifies

What are we trying to get?

"This attribute is very identifying" needs a number. We want Shannon entropy built from the idea of surprisal, its additivity, and the scale (33 bits) on which fingerprints are judged.

Definition 3.2.1 (Surprisal; entropy)**.** If an attribute takes value x_i with probability $P(x_i)$ in the population, observing x_i yields $-\log_2 P(x_i)$ bits of information (its *surprisal*): a value shared by half the population yields 1 bit, one shared by a thousandth yields ≈ 10 bits. The expected surprisal is the

Shannon entropy

$$H(X) = -\sum_i P(x_i) \log_2 P(x_i).$$

Low entropy: many users share the value; high entropy: the value is rare and narrows the candidate set more.

Reading the formula

Each bit halves the candidate set. Distinguishing one device among 7×10^9 needs $\log_2(7 \times 10^9) \approx 33$ bits. Independent attributes add: $H(X, Y) = H(X) + H(Y)$; correlated ones (OS and installed fonts) add less than their sum. A uniform attribute over n values has $\log_2 n$ bits, the maximum; a constant attribute has 0. The *distribution* matters as much as the number of values: the skewed time-zone example below has three values and less than one bit.

Worked example: Quiz 1, Q15, and a skewed variant

100 fingerprints: 50 Windows, 25 Linux, 25 macOS, so $P = (0.5, 0.25, 0.25)$:

$$H = 0.5 \log_2 2 + 0.25 \log_2 4 + 0.25 \log_2 4 = 0.5 + 0.5 + 0.5 = 1.5 \text{ bits}$$

(answer C; the maximum for three values is $\log_2 3 \approx 1.585$). Time zone with 80% UTC+1, 10% UTC+0, 10% UTC+2: $H = 0.8 \times 0.322 + 2 \times 0.1 \times 3.322 = 0.258 + 0.664 = 0.92$ bits: three values too, but far less identifying.

Hard questions, answered

Q. An attribute has 16 equally likely values; another has 1000 values, one of which covers 99% of users. Which identifies more on average?
A. First: $\log_2 16 = 4$ bits. Second: $0.99 \log_2(1/0.99) + 0.01 \log_2(999/0.01) \approx 0.014 + 0.166 = 0.18$ bits: the rare values are informative but rarely observed; entropy is an expectation over the population.

Q. Why report entropy rather than "number of distinct values"?
A. Identification power is information observed, which depends on how values are distributed, and attribute entropies add only when attributes are independent; counts capture neither.

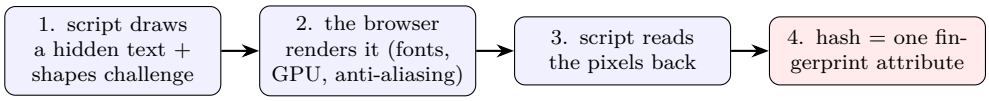
3.3 Attributes and Their Physics

What are we trying to get?

Each attribute on the syllabus has a reason it varies between devices. We want those reasons, because the examiner asks "how does canvas fingerprinting work" and "why can two identical laptops be told apart".

- **HTTP headers / User-Agent** (passive): grew from `NCSA_Mosaic/2.0 (Windows 3.1)` and `Mozilla/1.22 (compatible; MSIE 2.0; Windows 95)` into long strings naming engine, OS and versions; with `Accept`, `Accept-Encoding`, `Accept-Language`.

- **Screen, platform, time zone, cores, fonts, extensions** (active): e.g. `3840x2160x24`, `Win32`, UTC+8, 4 logical cores, the list of installed fonts.
- **Canvas** (Mowery & Shacham, W2SP 2012): JavaScript draws hidden text and shapes; the *instructions* are identical everywhere but the *rendered pixels* differ with browser, OS, fonts, GPU and graphics stack (anti-aliasing, hinting, sub-pixel rendering); the page reads the pixels back and hashes them. Quiz 1 Q14: render predefined content, read pixels, hash (B); it stores nothing in the canvas, retrieves no images, reads no serial number.
- **WebGL** vendor and renderer strings ("NVIDIA Corporation, GeForce GTX 4070 Ti").



- **GPU timing** (DrawnApart, Laor et al., NDSS 2022): two machines identical in model, OS, browser and GPU model still differ, because manufacturing variation makes some GPU execution units slightly faster. JavaScript measures timing traces of GPU work; a deep-learning model compresses each trace into an embedding; the same device yields similar embeddings despite noise, different devices yield different ones. On 2,550 devices and 370,392 fingerprints collected over 7 months in the wild, adding the GPU signal to FP-Stalker raised the median tracking duration in the 7-day revisit setting from 17.5 to 28 days (+10.5 days, ≈ 60%). The only *hardware-level* attribute on the syllabus; it improves *linkability*, not uniqueness.
- **Battery Status API** (first browser 2012; Olejnik, Acar, Castelluccia and Diaz, *The Leaking Battery*, 2015): charge level, charging state, estimated time remaining. A tracker on site A reads "73%, 14,237 s, not charging"; 20 seconds later on site B it reads "73%, 14,210 s, not charging". Few devices match that triple at that instant and the drift is predictable, so the two visits are linked *with no stored identifier*; minutes later the battery has drifted and the link expires. A temporary signal for short-term linking; the charge level could also be abused to raise prices.
- **IP address** (passive): WWW'20 study: 87% of devices kept the same IPv4 for more than 30 days (5,443 devices, 41,566 addresses), depending on country and ISP; the device–address graph was 93% unique; 20% of devices showed home–work–home cycles of 30 days or more. Shared behind NAT, so weak alone and strong in combination.
- **Mobile**: fingerprints are also highly unique (S&P'16), depending on vendor and model, through different attributes (User-Agent, emoji rendering, canvas).

Hard questions, answered

Q. Two laptops of the same model, OS, browser version and GPU model. Which attribute separates them and how?
A. DrawnApart: timing traces of GPU execution units reflect manufacturing variation invisible to every software attribute; a learned embedding of the trace is stable per device.

Q. Explain Battery-API linking and why it is not an identifier.

A. Level, time remaining and charging state at time t are rare in combination; at $t+20$ s the same device shows the same level and about 20 s less remaining, so a match is probable. Minutes later the triple has drifted and matches many devices: it links visits close in time only.

Q. Canvas needs JavaScript; the User-Agent does not. Why?
A. Canvas requires executing drawing code and reading pixels back (active); the User-Agent is a header the browser attaches to every request (passive).

3.4 Uniqueness, Linkability, Consistency

What are we trying to get?

Three research questions organise the field: can a fingerprint single you out, can it be followed as it changes, and do its attributes agree with each other. We want the numbers, the reason uniqueness collapsed between 2010 and 2018, and why that was not the end of tracking.

Uniqueness: how many are unique?
Panopticlick 470,161 fp, 94.2%
Beauty and the Beast (S&P'16) 118,934, 89.4%
Hiding in the Crowd (WWW'18) 1,816,776 desktop, 35.7%

Linkability: follow it as it evolves
FP-Stalker (S&P'18) 55 days avg
DrawnApart (NDSS'22) 17.5 → 28 days
Blink (SEAMS'15), FPRandom (ESSoS'17) break it

Consistency: do attributes agree?
FP-Scanner (USENIX Sec'18): spoofing tools are detectable and some leave per-user traces

Uniqueness fell from 94% to 36% between the early study and *Hiding in the Crowd* for three reasons: a far larger and more diverse sample (nearly two million desktop fingerprints on a popular site); the disappearance of high-entropy attributes (Flash, Java, enumerable plugin lists); and homogeneous populations on identical OS and browser versions. **Linkability** is about *evolution*: fingerprints change with updates, yet FP-Stalker follows a browser for 55 days on average with the right linking algorithm, and the GPU signal extends it; Blink breaks it by generating a random disposable environment (OS, browser, fonts) per session, FPRandom by adding controlled noise to canvas (adopted by Brave). **Consistency** cuts both ways: a tool that spoofs the User-Agent to say Windows while fonts, platform and WebGL still say Linux creates a detectable contradiction; FP-Scanner shows most such tools are detectable and some add a unique per-user trace, so a badly designed defence becomes a new attribute. Other works on the map: Fingerprinting in Style (USENIX Sec'21), Taming the Shape Shifter and Web Runner 2049 (DIMVA'20), Morellian analysis (DIMVA'19), FP-Redemption (DIMVA'21). `amiunique.org` (November 2014) shows your own fingerprint and, through an extension, its evolution.

Interpretation & discussion

36% unique is not the end of tracking. Non-unique fingerprints lie in *small* groups; add an IP address, a login or a short-term battery signal and the group collapses to one device; and following a changing fingerprint for weeks needs linkability, not global uniqueness. The same data has a defensive use: "does this login look familiar?" compares the current browser with past logins of a

known account; the fingerprint is a *risk signal*, never proof of identity (false positives, drift), so a mismatch should trigger step-up authentication rather than a denial or a silent acceptance.

Go deeper: the details an examiner asks about

- Quiz 1 Q11: Test 4 (all stores cleared; fingerprint 88 and profile 3 persist) is *consistent with* stateless relinking (B) and other browsers may share 88 (D); it proves neither a named identity (A) nor a failed clearing (C).
- Naive randomisation attacks linkability but creates inconsistency; a tool used by few people is itself rare. Brave's site-dependent noise (§3.6) is the designed answer.

Hard questions, answered

Q. A defence randomises canvas on every page load. Which property does it attack, and what can go wrong?
A. Linkability (and the canvas attribute's uniqueness). Risks: FP-Scanner-style coherence checks detect the tool; the tool's rarity becomes an identifying attribute; sites needing stable canvas output break.

Q. Why is 35.7% unique still a privacy problem?
A. Small anonymity groups collapse under one extra signal, and FP-Stalker-style linking follows a browser for weeks without uniqueness.

3.5 Blocking: Filter Lists and Behaviour

What are we trying to get?

Two philosophies of blocking: know the trackers in advance (lists) or recognise tracking behaviour as it happens. We want the list syntax the course tests, the four limitations of lists, and how Privacy Badger decides.

Definition 3.5.1 (Filter list; EasyList syntax)**.** A **filter list** is a maintained set of rules describing network requests or page elements that a blocker should block, allow, redirect or hide: the Disconnect list (Firefox, Brave), EasyList (ads) and EasyPrivacy (tracking) (AdBlock Plus, uBlock Origin). A network rule `||tracker.com^` blocks every request whose URL matches: `||` anchors at the start of a domain (so `sub.tracker.com` matches but `nottracker.com` does not), `^` marks the end of the domain or a separator such as `/`. A rule beginning with `@@` is an *exception* that allows something a broad rule would break.

Lists are *knowledge-based*: "we already know this domain tracks; if it matches, block". Their limitations: *coverage* (a tracker with no matching rule loads), *evasion* (trackers rotate domains, change URLs, or hide as first-party through CNAME), *breakage* (aggressive rules stop logins, video, payment, embeds), *maintenance* (continuous measurement, reports and updates). **Behaviour-based** protection asks what a third party *does*: does it appear across many websites, try to recognise the same browser, store identifying information? Privacy Badger (EFF)

watches for identifying cookies, local-storage "supercookies" and fingerprinting, and restricts or blocks a third party observed tracking across several unrelated sites; the threshold balances false positives on ubiquitous CDNs against coverage. FP-detect turns the tables: it detects which lists and extensions are active, so the blocker becomes an attribute.

Worked example: EasyList exercise

With `||amazon-adsystem.com^` in the list, both `/ad.js` and `/widgets/q?item=10` on that domain match the domain rule and are blocked *unless* an `@@` exception (for example on `/widgets/`) exists; exceptions exist precisely to prevent breakage, so one checks the list instead of guessing. "Block all of `ads.example.net` except `/consent/`": `||ads.example.net^` plus the exception `@@||ads.example.net/consent/`.

Hard questions, answered

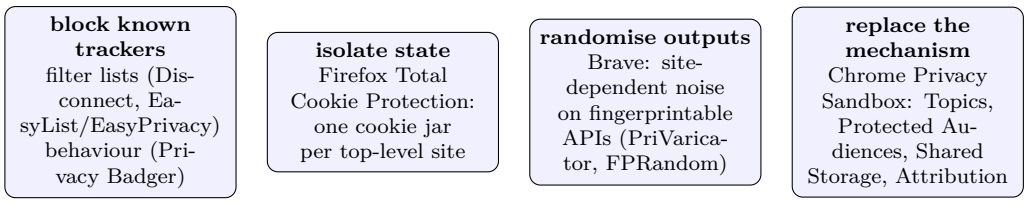
Q. A tracker is served from `static.publisher.com`. Which limitation is this and which protection can still catch it?
A. Evasion via first-party or CNAME hosting; URL lists miss it. DNS-level inspection (Firefox resolves the CNAME chain) or behaviour-based detection if it tracks across sites (it usually appears on one, so even Privacy Badger may miss it).

Q. Why can installing a blocker make you *more* fingerprintable?
A. Blockers change observable behaviour (which requests fail, which elements vanish); FP-detect infers the active lists and extensions, adding a rare attribute.

3.6 Browsers Against Tracking

What are we trying to get?

The browser vendors' responses differ in philosophy and in what they leave open. We want each mechanism, the timeline of the third-party-cookie saga, and the two evasion techniques (Topics misuse, CNAME cloaking) that show the limits.



- **Third-party cookie deprecation.** Google announced the end of third-party cookies in Chrome (January 2020): planned for Q2 2022, delayed to Q4 2023, then to Q4 2024, "the biggest change in ad tech since its inception"; in 2024 Chrome moved to a *user-choice* approach; in 2025–2026 Google confirmed no universal phase-out. The lesson: browser privacy is shaped by compatibility, advertising economics, standards and deployment, not only by technique. Advertisers meanwhile invested in their own user IDs, device data and first-party data (precise, easy to obtain, compatible with legislation),

and every platform opened a Conversions API (Meta, Amazon, Microsoft Offline Conversions, Google Enhanced Conversions, TikTok Events, X).

- **Privacy Sandbox** (Chromium): Topics API (interest-based targeting), Protected Audiences (retargeting), Shared Storage (frequency capping), Attribution Reporting. *Topics*: each week the browser derives 5 topics from the browsing history and adds one random topic, from a taxonomy of 469; the random topic gives deniability. Apple and Mozilla objected; PETS'24 showed that advertisers and publishers can abuse Topics to assign unique identifiers (the sequence of weekly topic sets observed by several sites is specific enough to link a browser) and that the design is "at odds with the natural diversity of user behaviours".
- **Firefox Total Cookie Protection**: each top-level site gets its own cookie jar; a tracker embedded on site A and site B receives two different cookies, so Theorem 2.4.1 fails and the tracker degrades to per-site analytics. It targets stateful cross-site tracking only: fingerprinting needs separate defences, CNAME-cloaked first-party cookies are untouched, and a sync between two trackers on the *same* site still works.
- **Brave**: add subtle, *site-dependent* randomness to selected fingerprintable outputs so repeated measurements are less useful as a stable identifier: the same site sees stable values (nothing breaks) while different sites see different values (no linking). Contrast with inconsistent spoofing (§3.4).
- **CNAME cloaking**: DNS maps `f0Ab1ze.mywebsite.com CNAME bad.tracker.com`, `bad.tracker.com A 119.183.2.123`. The browser sees a first-party hostname: cookies are first-party, URL filter lists miss it, and only resolving the CNAME chain (Firefox exposes DNS-level APIs to blockers) reveals the tracker. A threat to transparency because the mapping lives in DNS and server configuration; regulators such as CNIL must audit companies to see it.

Go deeper: the details an examiner asks about

- Dates: 2020 announcement; planned Q2 2022, Q4 2023, Q4 2024; 2024 user choice; 2025–26 no phase-out.
- Quiz 1 Q6 under Total Cookie Protection: (A) and (D) become false, two jars, two `tid` values.
- A browser needs all four columns of the figure: lists for known trackers, isolation for stateful reuse, randomisation for fingerprints, and an honest replacement for the advertising use case.

Hard questions, answered

Q. Why can Topics approximate an identifier although each week exposes only 5 of 469 topics plus a random one?
A. Several sites observe the same browser's weekly sets over many weeks; the *sequence* of sets is highly specific to one history, so the sites link the browser by matching sequences; one random topic per week is too little noise across weeks.

Q. Why is CNAME cloaking "a big threat for transparency"?
A. Nothing in the page reveals the tracker: the hostname is first-party and

the alias lives in DNS; users and URL-based blockers see a first-party request; only DNS inspection or an audit of the company exposes it.

Q. Compare Firefox partitioning and Brave randomisation: which threat does each address, and what does each miss?
A. Firefox: stateful cross-site tracking through cookie reuse; misses fingerprinting, CNAME-cloaked first-party cookies, same-site syncing. Brave: fingerprint stability; misses stateful tracking unless combined with blocking or partitioning.

Chapter takeaways

Summary: Stateless tracking and defences

- Stateless = recompute from exposed properties; passive, active, behavioural; a hash makes one unreadable identifier of a *browser*, not a person.
- $H = -\sum P \log_2 P$; independent attributes add; 33 bits single out a device; skew lowers entropy.
- Attributes: headers; screen, fonts, time zone; canvas rendering; WebGL; GPU timing (hardware); battery (short-term); IP (stable for weeks).
- Uniqueness fell to 36% on desktop; linkability (55 days; 28 with GPU) is the real threat; inconsistent spoofing is detectable.
- Lists (`||domain^`, `@@`) are knowledge-based and evadable (CNAME); Badger is behaviour-based; Firefox partitions, Brave randomises per site; Topics is a quasi-identifier; Chrome kept third-party cookies.